Milepæl 2: Point forecast med TCN

Mål:

Implementere og trene en første TCN deep learning modell som bruker 28 dagers historikk til å predikere 7 dagers etterspørsel frem i tid. Modellen skal trenes på treningssettet og evalueres på validerings- og testsettet med forecast-metrikker, først og fremst MAE.

Målet med milepælen er å få en fungerende TCN-basert point forecast-modell som kan sammenlignes direkte med baseline-modellen fra milepæl 1.

Bygg:

- Konvertere X_train, y_train, X_val, y_val, X_test og y_test til PyTorch-tensorer
- En enkel TCN-modell med inputvindu på 28 og output på 7 (dager)
- Treningsløype med loss-funksjon, optimizer og flere epochs
- Kjøre modellen på treningsdata og overvåke trening/validering
- Generere punktprognoser (point forecasts) på testssettet
- Beregne MAE
- Sammenligne TCN-modellen med baseline-modellen fra milepæl 1

Ønskelig output:

- En fungerende TCN-model som tar inn tensorer med shape (batch_size, 1, 28) og returnerer outputs med shape (batch_size, 7)
- Trening som faktisk kjører over flere epochs
- Forecast-prediksjon på testsettet
- MAE-verdi for TCN-modellen
- En første sammenligning av TCN og naive baseline

Resultat:

En TCN-modell som trenes med train/val/test-split, der beste modell velges fra validering og test MAE sammenlignes mot baseline i tillegg til at inventory metrics regnes ut og sammenlignes for testprediksjonene for resultatene fra både baseline og TCN.

“End-to-end point forecast run” med:

- hyperparametere
- best val-loss: 0.7241
- test MAE rå: 0.7366
- test MAE avrundet: 0.7016
- baseline MAE: 0.9488
- eksempelprediksjoner
- inventory-metrics:

	total cost	stockout rate	fill rate
Baseline	5673	0.296	0.320
TCN	6777	0.414	0.036

```
jakob@158-37-236-3 project % /usr/bin/python3 /Users/jakob/Desktop/2026/DAT255/project/src/train.py
X_train shape: torch.Size([1315, 1, 28])
y_train shape: torch.Size([1315, 7])
X_val shape: torch.Size([282, 1, 28])
y_val shape: torch.Size([282, 7])
X_test shape: torch.Size([282, 1, 28])
y_test shape: torch.Size([282, 7])
Device: cpu
Epoch  1/60 | train_loss=0.8041 | val_loss=0.7338
Epoch 10/60 | train_loss=0.7308 | val_loss=0.7387
Epoch 20/60 | train_loss=0.6499 | val_loss=0.7744
Epoch 30/60 | train_loss=0.5990 | val_loss=0.8010
Epoch 40/60 | train_loss=0.5371 | val_loss=0.8333
Epoch 50/60 | train_loss=0.5036 | val_loss=0.8726
Epoch 60/60 | train_loss=0.4743 | val_loss=0.9026

Testresultat fra beste valideringsmodell (epoch 5/60).
Beste val-loss: 0.7241
TCN test MAE (rå): 0.7366
TCN test MAE (avrundet): 0.7016
Baseline test MAE: 0.9488

Første TCN-prediksjon (rå): [ 0.068 -0.001  0.052  0.002 -0.    -0.009 -0.004]
Første TCN-prediksjon (avrundet): [0 0 0 0 0 0 0]
Første baseline-prediksjon: [0 1 0 0 1 0 0]
Første faktisk target: [3 1 0 0 1 0 2]

Inventory metrics på testsett:
TCN total cost: 6777.00, stockout rate: 0.414, fill rate: 0.036
Baseline total cost: 5673.00, stockout rate: 0.296, fill rate: 0.320
jakob@158-37-236-3 project %
```

Hva kan vi tolke fra disse resultatene?

Forecast-delen ser lovende ut, men decision-delen er foreløpig dårlig.

- Train loss går gradvis ned fra 0.80 til 0.47
- TCN test MAE er 0.74, mens Baseline test MAE er 0.95.

TCN slår altså baseline ganske klart på forecast-accuracy.

Men samtidig er:

- total cost for TCN: 6777 mot baseline: 5673
- stockout rate TCN: 0.414 mot baseline: 0.296
- fill rate TCN: 0.036 mot baseline: 0.320

altså er baseline mye sterkere på decision accuracy ved inventory simulation.

Modellen blir altså flinkere til å huske treningsdata, men blir dårligere til å generalisere valideringsdata.

Vi ser også tydelig overfitting, ved at val loss starter på laveste verdi og blir gradvis verre, hvilket betyr at modellen lærer treningsdata bedre og bedre, men generaliserer dårligere etter tidlige epoker.

Resultatene viser dermed at lavere forecast-feil ikke nødvendigvis gir bedre beslutningskvalitet i lagerstyring. Dette motiverer neste steg i prosjektet, hvor vi vil undersøke uncertainty-aware forecasting i Milepæl 3.